C30 Pro / Jieli BLE

Reverse Engineering Completo

Sesiones 1, 2 y 3 combinadas

Chip: Jieli (杰理科技) | Protocolo AB | BTSnoop Analysis

1. Identificación del Hardware

1.1 Dispositivo

Campo	Valor
Modelo	C20 Pro / C30 Pro (mismo hardware)
App oficial	FitHere (Google Play)
Chip BLE real	Jieli (杰理科技) - identificado via jadx-gui en APK
MAC usada en pruebas	F4:FE:A1:C8:FA:AE

1.2 Teardown del hardware (Reloj 2 - abierto)

La PCB fue fotografiada y los chips identificados visualmente:

Componente	Identificacion	Funcion
SoC principal	PHY6222 (Phyplus)	ARM Cortex-M0, BLE 5.0 integrado
Chip secundario	Sensor HR/SpO2	Frecuencia cardiaca / oximetro
LP4054	Cargador LiPo	Gestion de bateria

⚠ IMPORTANTE: El chip identificado en el teardown es PHY6222 (Phyplus), pero el protocolo BLE es Jieli. Esto sugiere que el reloj 1 (con bateria hinchada) y el reloj 2 (abierto fisicamente) pueden ser variantes diferentes del mismo modelo, o que el firmware Jieli corre sobre hardware PHY6222.

⚠ Bateria del reloj 1: LiPo severamente hinchada. Riesgo de thermal runaway. DESCARTADO para uso.

SDK del PHY6222 disponible publicamente:
`github.com/PHYTEC-Microcontrollers/PHY6222`

2. Identificacion del protocolo BLE

2.1 UUIDs Jieli (del APK FitHere - BleUUID.java)

```
// Perfil Jieli (activo en C30_Pro)
JIELI_SERVICE_UUID  = "0000ae00-0000-1000-8000-00805f9b34fb"
JIELI_WRITE_UUID    = "0000ae01-0000-1000-8000-00805f9b34fb"
JIELI_NOTIFY_UUID   = "0000ae02-0000-1000-8000-00805f9b34fb"

// Perfiles alternativos definidos pero NO usados por este reloj:
TFIT_WRITE_UUID     = "00002f01-0000-1000-8000-00805f9b34fb"
LUFUN_WRITE_UUID    = "00002d01-0000-1000-8000-00805f9b34fb"
```

⚠ Los UUIDs ae00/ae01/ae02 NO aparecen en el dump GATT en modo normal. Solo son visibles cuando FitHere ya establecio la sesion. Ver seccion 5.

2.2 Caracteristicas GATT visibles (modo normal, sin FitHere)

Handle	UUID	Properties	Notas
0x0012	0000ff02	0x08 Write	
0x0017	0000ff04	0x0a Write+Read	
0x0031	0000ffd1	0x04 Write WR	
0x004f	0000fee2	0x04 Write WR	
0x0054	0000fee5	0x04 Write WR	
0x0056	0000fee6	0x04 Write WR	
0x0063	0000fd03	0x0c Write+Notify	CANDIDATO PRINCIPAL
0x0065	0000fd04	0x10 Notify	Respuestas / notificaciones
0x0046	0000fea1	0x12 Read+Notify	
0x004c	0000fee1	0x12 Read+Notify	Posible trigger unlock

3. Analisis BTSnoop - Sesion 3 (NUEVO)

3.1 Metadatos de la captura

Campo	Valor
Formato	BTSnoop v1, HCI UART (H4)

Campo	Valor
Tamano	~10 MB
Total paquetes	41,529
Duracion	29.45 minutos (19:57 a 20:26 UTC)
Handles activos	`0x0033 (51) y 0x0200 (512)`

3.2 Las dos conexiones simultaneas

Handle	Tipo BT	CIDs principales	Direccion	Paquetes
`0x0033 (51)`	BR/EDR Classic	`0x03ad / 0x03ac`	W->H streaming	~8,675
`0x0200 (512)`	BLE	`0x0017 (dinamico)`	Bidireccional	~5,535

Distribucion de flags HCI:

Flag	Descripcion	Paquetes
0 (W->H data)	ACL data recibida del controlador (del reloj)	13,968
1 (H->W data)	ACL data enviada al controlador (al reloj)	242
2 (Event)	Eventos HCI	340
3 (Command)	Comandos HCI	26,979

3.3 Handle 0x0033 - BR/EDR Classic (streaming de sensores)

Canal L2CAP dinamico con CIDs 0x03ac y 0x03ad. Caracteristicas:

- ~943 bytes por paquete, siempre del reloj al host (unidireccional)
- Frecuencia: ~29.45 fps (cada ~34ms)
- Stream continuo: muy probablemente datos de acelerometro/giroscopio en tiempo real
- La estructura de los datos no ha sido decodificada completamente

Muestra de payload (primeros bytes):

```
420080e08c690231a4000000000247fc0000b090800300ffffff97211b14e5fc2e...
```

3.4 Handle 0x0200 - BLE (canal de comandos)

3.4.1 CIDs utilizados

CID	Paquetes	Funcion
`0x0001`	3	L2CAP signaling
`0x0005 / 0x0006 / 0x0007 / 0x0008`	varios	Negociacion de canales L2CAP
`0x0009`	9	Respuestas cortas del reloj (status/RSSI)

CID	Paquetes	Funcion
`0x000e`	6	Handshake principal (16 bytes)
`0x0017`	5,186 (W->H) + 116 (H->W)	CANAL PRINCIPAL DE DATOS

3.4.2 El canal 0x0017 NO es GATT estandar

⚠ El CID 0x0017 es un canal L2CAP dinamico propietario, NO es ATT (CID 0x0004). Esto explica por que gatttool no funcionaba: se intentaba escribir via GATT cuando el protocolo real usa un canal completamente diferente.

3.4.3 Comandos Host -> Reloj (116 paquetes, CID 0x0017)

Patron de los comandos:

```
04 00 1b 2f 00 5a [CMD] [SUB] [DATA...padding...]
```

Donde:

- 04 00 = L2CAP length (4 bytes de payload util + header)
- 1b 2f 00 = constante desconocida
- 5a = byte magico ('Z' en ASCII)
- [CMD] = codigo de comando
- [SUB] = subcomando
- [DATA] = datos del comando (rellenado a 25 bytes total)

Comandos identificados (frecuencia en la captura):

CMD	SUB	Veces	Hipotesis
`0x02`	`0x03/0x0e/0x14/0x1a/0x35/0x3b`	3 c/u	Configuracion multiple
`0x05`	`0x03/0x21/0x28`	3-5	Init/setup
`0x06`	`0x04`	6	Sync / ping
`0x06`	`0x0b/0x0c`	5	Config pantalla?
`0x07`	`0x08`	5	Desconocido
`0x08`	`0x02/0x46`	3-5	Datos usuario
`0x09`	`0x06/0x25`	3-5	Configuracion
`0x0c`	`0x05`	15	MAS FRECUENTE - config display?
`0x0e`	`0x11`	10	Configuracion hora/notif
`0x0f`	`0x11/0x15`	5	Desconocido
`0x10`	`0x48`	3	Desconocido
`0x12`	`0x13`	5	Desconocido
`0x14`	`0x00`	3	Desconocido

3.4.4 Respuestas Reloj -> Host (5,070 paquetes, CID 0x0017)

El reloj envia la gran mayoria del trafico usando el comando 0x29 = transferencia bulk:

Estructura de cada paquete 0x29:

```
04 00 52 2c 00 ab 29 [SEQ_LO] [SEQ_HI] [18 bytes de datos]
```

Donde:

- 04 00 = L2CAP length
- 52 2c 00 ab = bytes de MAC del dispositivo (parcial, AB:00:2C:52)
- 29 = comando bulk transfer
- [SEQ_LO] [SEQ_HI] = numero de secuencia 16-bit little-endian
- 18 bytes = datos de salud / configuracion

⚠ Se identificaron 2,513 sesiones de transferencia distintas durante los 29 minutos de captura. El reloj envia datos constantemente hacia el host.

3.5 Respuestas cortas del reloj (CID 0x0009)

Solo aparece 9 veces, siempre con el mismo payload:

```
0400522c00ab 06 28 06 9a3e
```

Posiblemente: echo del comando 0x06/0x28 con valor de respuesta 0x06 y checksum 0x9a3e

3.6 Handshake principal (CID 0x000e)

Aparece 3 veces durante la sesion (en cada reconexion):

```
0400522c00ab 0b 04 01 1a 03 0f [COUNTER] [CRC]
```

El byte [COUNTER] incrementa en cada sesion (0x0e, 0x0f, 0x10...), lo que sugiere un contador de sesion.

4. Protocolo AB - Estructura Completa

4.1 Formato de paquete AB

```
+--------+-------+-------+-------------+---------+
| 0xAB   | LEN   | CMD   | DATA ...    | CRC8    |
+--------+-------+-------+-------------+---------+
  1 byte  1 byte  1 byte   variable      1 byte
```

Campo	Descripcion
`0xAB`	Header fijo, siempre presente
LEN	Longitud total del paquete en bytes
CMD	Codigo de comando
DATA	Payload variable segun el comando
CRC8	Checksum del paquete completo

4.2 Secuencia de handshake/init completa

Orden exacto observado en el pcap (debe enviarse en este orden):

```
ab 05 39 00 97   # CMD 0x39 - desconocido
ab 05 51 00 bb   # CMD 0x51 - desconocido
ab 05 31 01 bf   # CMD 0x31 sub:01
ab 05 06 00 a2   # CMD 0x06 - sync/ping
ab 04 00 0c      # CMD 0x00 (4 bytes)
ab 04 03 ee      # CMD 0x03 (4 bytes)
ab 04 70 f4      # CMD 0x70 (4 bytes)
ab 0b 04 01 1a 03 0f 02 09 24 d8   # HANDSHAKE PRINCIPAL
ab 05 21 02 b1   # CMD 0x21 sub:02
ab 05 0c 00 45
ab 05 0b 00 2b
ab 08 02 00 00 00 00 ec
ab 05 08 00 7e
ab 05 25 00 36
ab 05 05 00 f7
ab 05 13 06 6c
ab 05 15 06 c6
ab 05 11 01 7e
ab 05 11 02 9c
ab 05 11 04 41
```

4.3 Desglose del handshake principal

```
ab 0b 04 01 1a 03 0f 02 09 24 d8

AB   = header
0b   = LEN (11 bytes total)
04   = CMD
01   = subcmd
1a 03 = posible version de firmware
0f   = desconocido
02 09 24 = posible timestamp parcial / ID sesion
d8   = CRC8
```

4.4 Comando CMD 0x29 - Transferencia bulk (reloj -> host)

El reloj usa este comando para enviar grandes volumenes de datos al host:

```
Formato en L2CAP (CID 0x0017):
04 00 52 2c 00 ab 29 [SEQ16_LE] [18 bytes datos]

Total: 25 bytes por paquete
Secuencias: numeracion de 16 bits, incrementa por sesion de datos
Sesiones identificadas: 2,513 en 29 minutos de captura
```

5. El servicio ae00 oculto

5.1 Comportamiento observado

El servicio ae00 (con handles altos 0x2c52, 0x3012) NO aparece en el dump GATT estandar. Solo es visible cuando:

- FitHere esta conectado y activo
- El reloj ha recibido el handshake/unlock previo
- La sesion fue capturada despues de que FitHere completo la inicializacion

5.2 Handles en sesion activa (del BTSnoop)

Handle	Funcion
`0x2c52`	Escritura de comandos AB al reloj
`0x3012`	Enable notify (escribir 0x0100 aqui)

5.3 Hipotesis del unlock

FitHere probablemente escribe en fd03 (handle 0x0063) o fee1 (handle 0x004c) durante la inicializacion, lo que desbloquea el servicio ae00.

⚠ El pcap fue capturado DESPUES del handshake inicial, por lo que no captura el trigger de unlock. Capturar desde t=0 de la conexion es el siguiente paso critico.

6. Comandos para inyeccion

6.1 Con gatttool

```
# Conectar al dispositivo
gatttool -b F4:FE:A1:C8:FA:AE -I
[LE]> connect

# Habilitar notificaciones en fd04
char-write-req 0x0066 0100

# Enviar secuencia de init en orden
char-write-cmd 0x0063 ab05390097
char-write-cmd 0x0063 ab055100bb
char-write-cmd 0x0063 ab053101bf
char-write-cmd 0x0063 ab050600a2
char-write-cmd 0x0063 ab04000c
char-write-cmd 0x0063 ab0403ee
char-write-cmd 0x0063 ab0470f4
char-write-cmd 0x0063 ab0b04011a030f020924d8
char-write-cmd 0x0063 ab052102b1
char-write-cmd 0x0063 ab050c0045
char-write-cmd 0x0063 ab050b002b
char-write-cmd 0x0063 ab080200000000ec
char-write-cmd 0x0063 ab0508007e
char-write-cmd 0x0063 ab05250036
char-write-cmd 0x0063 ab050500f7
char-write-cmd 0x0063 ab0513066c
char-write-cmd 0x0063 ab051506c6
char-write-cmd 0x0063 ab0511017e
char-write-cmd 0x0063 ab0511029c
```

```
char-write-cmd 0x0063 ab05110441
```

⚠ PROBLEMA CONOCIDO: Handle 0x0063 (fd03) esta en el GATT normal, pero los comandos no producen respuesta. El reloj requiere que FitHere establezca la sesion primero para desbloquear el canal ae00 real (handles 0x2c52/0x3012).

6.2 Script Python con bleak

```
# pip install bleak
import asyncio
from bleak import BleakClient

MAC = 'F4:FE:A1:C8:FA:AE'
FD03_UUID = '0000fd03-0000-1000-8000-00805f9b34fb'
FD04_UUID = '0000fd04-0000-1000-8000-00805f9b34fb'

INIT_SEQUENCE = [
    bytes.fromhex('ab05390097'),
    bytes.fromhex('ab055100bb'),
    bytes.fromhex('ab053101bf'),
    bytes.fromhex('ab050600a2'),
    bytes.fromhex('ab04000c'),
    bytes.fromhex('ab0403ee'),
    bytes.fromhex('ab0470f4'),
    bytes.fromhex('ab0b04011a030f020924d8'),
    bytes.fromhex('ab052102b1'),
    bytes.fromhex('ab050c0045'),
    bytes.fromhex('ab050b002b'),
    bytes.fromhex('ab080200000000ec'),
    bytes.fromhex('ab0508007e'),
    bytes.fromhex('ab05250036'),
    bytes.fromhex('ab050500f7'),
    bytes.fromhex('ab0513066c'),
    bytes.fromhex('ab051506c6'),
    bytes.fromhex('ab0511017e'),
    bytes.fromhex('ab0511029c'),
    bytes.fromhex('ab05110441'),
]

def notification_handler(sender, data):
    print(f'NOTIFY from {sender}: {data.hex()}')

async def main():
    async with BleakClient(MAC) as client:
        print(f'Connected: {client.is_connected}')
        await client.start_notify(FD04_UUID, notification_handler)
        for cmd in INIT_SEQUENCE:
            print(f'Sending: {cmd.hex()}')
            await client.write_gatt_char(FD03_UUID, cmd, response=False)
            await asyncio.sleep(0.1)
        await asyncio.sleep(5)

asyncio.run(main())
```

7. Estado del proyecto

7.1 Confirmado

✓ Chip hardware (teardown reloj 2): PHY6222 (Phyplus), ARM Cortex-M0, BLE 5.0
✓ Chip firmware/protocolo: Jieli (杰理科技)
✓ App oficial: FitHere
✓ Protocolo: AB header (0xAB + LEN + CMD + DATA + CRC8)
✓ Canal L2CAP real: CID 0x0017 dinamico (NO es GATT/ATT estandar)
✓ Handle de escritura en sesion activa: 0x2c52
✓ Handle de notify en sesion activa: 0x3012
✓ Secuencia de init completa capturada del trafico real
✓ 2,513 sesiones de transferencia de datos identificadas en BTSnoop
✓ Stream BR/EDR de sensores: ~29.45 fps, 943 bytes/paquete
✓ Byte magico del protocolo: 0x5a ('Z')

7.2 Pendiente / sin resolver

⚠ El servicio ae00 (handles 0x2c52/0x3012) no aparece en GATT estandar. Requiere trigger previo no capturado.
⚠ Bateria hinchada en reloj 1 - descartado. Solo queda reloj 2 abierto fisicamente.
⚠ Pads SWD/UART del PHY6222 no identificados aun en la PCB del reloj 2.
⚠ Firmware del PHY6222 potencialmente writeprotected - requiere verificacion.

7.3 Proximos pasos recomendados

- Capturar BTSnoop desde t=0 de conexion de FitHere (antes del primer sync) para identificar el trigger de unlock del servicio ae00
- Fotografiar el reverso de la PCB del reloj 2 para identificar pads SWD
- Intentar conectar SWD con ST-Link V2 al PHY6222 si se encuentran los pads
- Buscar en jadx (FitHere_src/) el codigo del unlock/pair inicial
- Explorar SDK publico PHY6222 de Phyplus en GitHub

8. Referencias

Recurso	Detalle
Protocolo Jieli BLE	No documentado publicamente
SDK PHY6222	github.com/PHYTEC-Microcontrollers/PHY6222
SDK interno APK	com.tjd.lefun.sdk.device.*
APK analizado	FitHere (Google Play)
MAC del dispositivo	F4:FE:A1:C8:FA:AE (redactar antes de publicar)
jadx-gui	Decompilacion APK
gatttool / bleak	Interaccion BLE
Python BTSnoop parser	Analisis del pcap (41,529 paquetes, 29.45 min)

Recurso	Detalle
Android HCI Snoop Log	Captura BLE desde dispositivo Android

Documento generado de 3 sesiones de ingenieria inversa - para continuacion en GitHub